Project Report

on

“CivicSense: Automated Build, Test and Zero-Downtime Deployment with a Jenkins Pipeline as Code”

Submitted by

Mohammad Iqbal Patan

UID: 24MCC20081

MCA (CC & DevOps) · Semester 3 · Section 102/A

Under the guidance of

Mr. Danish Rashid Pala

Subject: DevOps Process Automation Lab | Subject Code: 25CAP-732

Class Group: 25CAP-732:P::GP-A

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS
CLOUD COMPUTING & DEVOPS

University Institute of Computing (UIC)

Chandigarh University

October 2026

TABLE OF CONTENTS

Certificate **6**
Declaration **7**
Acknowledgement **8**
Abstract **9**
1. Introduction **10**
1.1 Background 10
1.2 Problem Statement 10
1.3 Proposed Solution 11
1.4 Scope of the Project 11
1.5 Organisation of the Report 12
2. Background and Project Objectives **13**
2.1 DevOps and the Delivery Pipeline 13
2.2 Testing in DevOps and Continuous Testing 13
2.3 Version Control and Branching Strategy 13
2.4 Jenkins 14
2.5 Continuous Integration, Delivery and Deployment 14
2.6 Deployment Strategies 14
2.7 Configuration Management 15
2.8 Project Objectives 15
3. System Requirements **16**
3.1 Hardware Requirements 16
3.2 Software Requirements 16
3.3 Functional Requirements 16
3.4 Non-Functional Requirements 17
4. Tools and Technologies Used **18**
4.1 Jenkins and its Plugins 18
4.2 Docker as the Build Environment 19
4.3 GitHub Actions 19
5. System Design **20**
5.1 Pipeline Architecture 20
5.2 Jenkins Configured as Code 20
5.3 Branching Workflow 21
5.4 Environments and Blue-Green Deployment 21
5.5 Configuration Management Design 22
5.6 Security of the Pipeline 22
6. Implementation Steps **24**
6.1 Step 1: Start the Repository from the Application History 24
6.2 Step 2: Make the Release Visible 24
6.3 Step 3: Static Analysis and the Tools Image 24
6.4 Step 4: Tests Inside the Image, on MySQL 24
6.5 Step 5: Dependency Audit and Image Smoke Test 24

6.6 Step 6: The Environments and the Blue-Green Script....24
6.7 Step 7: The Deployment Smoke Test....25
6.8 Step 8: The Jenkinsfile....25
6.9 Step 9: Jenkins as Code....25
6.10 Step 10: GitHub Actions and the Branching Workflow....25
7. Implementation and Code Explanation....26
7.1 Project Structure....26
7.2 The Jenkinsfile....26
7.3 Jenkins Configuration as Code....28
7.4 The Blue-Green Deployment Script....29
7.5 The Environment Description....30
7.6 The Smoke Test....31
7.7 Tests on MySQL Inside the Image....32
8. Testing and Results....33
8.1 Testing Strategy....33
8.2 Pipeline Runs....33
8.3 Stage Durations....34
8.4 Static Analysis and Dependency Audit....35
8.5 Unit Tests and Coverage....36
8.6 Branching Workflow and a Test-First Change....36
8.7 Zero-Downtime Releases....37
8.8 Automatic Rollback....38
8.9 Manual Rollback....39
8.10 Lead Time and Feedback....40
8.11 Summary of Results....40
8.12 Defects Found and Fixed....41
9. Screenshots....42
10. Conclusion....46
11. Future Scope....47
References....48
Appendix A: Running and Demonstrating the Project....49

LIST OF FIGURES

S. No.	Figure	Page
1	Figure 5.1 – The delivery pipeline: GitHub Actions and the Jenkins multibranch pipeline	20
2	Figure 5.2 – Jenkins rebuilt from files: plugins, configuration and the job	21
3	Figure 5.3 – Feature branch, pull request and merge into main	21
4	Figure 5.4 – One environment: two web slots behind nginx, and the steps of a release	22
5	Figure 8.1 – Jenkins stage view of the main branch: every stage of every run with its duration	35
6	Figure 8.2 – Build #6: static analysis, 41 tests on MySQL 8.4, dependency audit and image smoke test	35
7	Figure 8.3 – Coverage report archived by build #6	36
8	Figure 8.4 – The test first (red in Jenkins), then the change (green in Jenkins and on the pull request)	37
9	Figure 8.5 – Build #4: blue-green release to production and the smoke test	38
10	Figure 8.6 – Requests counted during each production release	38
11	Figure 8.7 – Build #7: the login check fails after the switch and production rolls back by itself	39
12	Figure 8.8 – Manual rollback and roll forward under traffic	40
13	Figure 9.1 – Jenkins dashboard: the multibranch job created from casc.yaml, with its two branches	42
14	Figure 9.2 – Pipeline graph of build #7: parallel stages, and the failed production deployment	42
15	Figure 9.3 – JUnit test results of a main build: 41 tests	43
16	Figure 9.4 – The feature branch job: the failing test-first build and the passing build	43
17	Figure 9.5 – Pull request #1 on GitHub with its checks, merged into main	44
18	Figure 9.6 – GitHub Actions runs of the repository	44
19	Figure 9.7 – Production portal after build #9: no demonstration data, and the footer names the release and slot that served the page	45

LIST OF TABLES

S. No.	Table	Page
1	Table 2.1 – Continuous integration, delivery and deployment	14
2	Table 2.2 – Deployment strategies	14
3	Table 3.1 – Hardware requirements	16
4	Table 3.2 – Software requirements	16
5	Table 3.3 – Functional requirements of the pipeline	16
6	Table 4.1 – Tools and technologies used	18
7	Table 5.1 – Quality gates of the pipeline	20
8	Table 5.2 – Where each kind of configuration lives	22
9	Table 5.3 – Security measures in the pipeline	22
10	Table 8.1 – All pipeline runs	33
11	Table 8.2 – Stage durations of the six normal releases	34
12	Table 8.3 – GitHub Actions runs	37
13	Table 8.4 – Requests during production releases	38
14	Table 8.5 – Delivery measurements in the style of the DORA metrics	40
15	Table 8.6 – Summary of measured results	40
16	Table 8.7 – Defects found and fixed	41
17	Table A.1 – Accounts	49

CERTIFICATE

This is to certify that the project report entitled **"CivicSense: Automated Build, Test and Zero-Downtime Deployment with a Jenkins Pipeline as Code"** submitted by **Mohammad Iqbal Patan** (UID: 24MCC20081), MCA (Cloud Computing & DevOps), Semester III, Section 102/A (class group 25CAP-732:P::GP-A), in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (Cloud Computing & DevOps) at Chandigarh University, is a record of bonafide work carried out by the student under my supervision and guidance as part of the course **DevOps Process Automation Lab (Subject Code: 25CAP-732)** during the academic session 2026–2027.

The project involved building a complete delivery pipeline for a real Django and MySQL application: a declarative Jenkins pipeline with parallel quality gates, tests on the production image against MySQL, a feature-branch workflow checked by Jenkins and GitHub Actions, blue-green deployment to staging and production with automatic rollback, configuration management, and a Jenkins server defined entirely as code. It demonstrates the practical application of DevOps process automation to a real-world problem.

To the best of my knowledge, the work embodied in this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Mr. Danish Rashid Pala
Project Guide
University Institute of Computing

Head of Department
University Institute of Computing
Chandigarh University

Date: _____

Place: Chandigarh University, Mohali, Punjab

DECLARATION

I, **Mohammad Iqbal Patan**, student of Master of Computer Applications (Cloud Computing & DevOps), Semester III, University Institute of Computing, Chandigarh University, hereby declare that the project report entitled **"CivicSense: Automated Build, Test and Zero-Downtime Deployment with a Jenkins Pipeline as Code"**, submitted for the course DevOps Process Automation Lab (25CAP-732), is an authentic record of my own work carried out under the guidance of Mr. Danish Rashid Pala.

All the code, configuration, measurements and screenshots presented in this report were produced while building, running and testing the project. Wherever the work of others has been used, it has been duly acknowledged in the text and listed in the references.

This report has not been submitted to any other university or institution for the award of any degree or diploma.

Date: _____
Place: Mohali, Punjab

Mohammad Iqbal Patan
UID: 24MCC20081

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project titled **“CivicSense: Automated Build, Test and Zero-Downtime Deployment with a Jenkins Pipeline as Code”**.

First and foremost, I would like to thank **Mr. Danish Rashid Pala**, my project guide, for the valuable guidance, continuous support and encouragement throughout the course of this project. The suggestions and feedback I received played a key role in shaping both the pipeline and this report.

I extend my heartfelt thanks to the faculty members of the University Institute of Computing, Chandigarh University, for providing the facilities, the curriculum in DevOps, process automation and cloud computing, and a stimulating environment in which to carry out this work.

I am also grateful to the open-source communities behind Jenkins, Git, Docker, Django, MySQL and nginx. Their freely available software and documentation made it possible to build and operate a production-style delivery pipeline on a single laptop.

Lastly, I would like to thank my family, friends and classmates for their motivation, moral support and constructive feedback during the development of this project.

Mohammad Iqbal Patan
UID: 24MCC20081

ABSTRACT

Releasing software by hand is slow and error-prone: tests are skipped, releases take the site down, and a bad release stays live until someone notices it and repairs it manually. DevOps process automation replaces this with a delivery pipeline that every change passes through. This project builds such a pipeline for **CivicSense**, a civic complaint platform written with Django and MySQL, and measures how well it works.

A **declarative Jenkinsfile** runs static analysis (flake8, bandit) beside the image build, then **41 tests inside the production image against MySQL 8.4** beside a dependency audit (pip-audit), and a smoke test of the image. A **multibranch** job builds every Git branch; only `main` is deployed, first to staging and then to production, using a **blue-green** switch: the release starts beside the live version, is checked, and nginx is then reloaded gracefully. If the smoke test fails after the switch, the previous release is restored automatically. Staging and production share one Compose description and differ only in small environment files; secrets are generated per environment outside Git. The **Jenkins controller itself is configured as code** – plugins, security and the job come from files in the repository – and **GitHub Actions** checks every pull request independently.

Eleven real pipeline runs were measured. A normal release took 167–223 s and reached production 209–260 s after the push, without a manual step. Statement coverage is 82 %, pip-audit found no known vulnerabilities, and a bandit finding led to non-guessable complaint tracking ids. During production releases one request failed (a stylesheet briefly deleted by `collectstatic --clear`); after the fix, 23,472 requests sent during five releases all succeeded. A failing test on a feature branch stopped the branch in Jenkins and on the pull request, and a faulty production configuration was caught by a real login in the smoke test and rolled back in 0.4 s. Running the pipeline also exposed defects that the developer machine never showed, including scripts that lost their executable bit on an NTFS drive and an nginx reload that returns before it takes effect.

The source code, the pipeline and this report are available at https://github.com/iPatan7/civicsense-devops.

Keywords: DevOps, Continuous Integration, Continuous Deployment, Jenkins, Jenkinsfile, Configuration as Code, Multibranch Pipeline, Git Branching, GitHub Actions, Blue-Green Deployment, Rollback, Continuous Testing, Docker.

CHAPTER 1: INTRODUCTION

1.1 Background

Writing an application is only half of software engineering. The other half is getting every change from a developer's machine to the users safely, quickly and again and again. For a long time this second half was done by hand: someone ran the tests (or forgot to), built a package, copied it to a server, stopped the old version, ran the database migrations and started the new version, usually late in the evening because the site would be down for a few minutes.

DevOps is the set of practices that removes this gap between development and operations. Its central tool is the **delivery pipeline**: an automated sequence of steps that every change goes through. *Continuous integration* (CI) merges and tests every change as soon as it is pushed; *continuous delivery* keeps every passing build ready to release; *continuous deployment* releases it automatically. Around the pipeline sit version control with a branching strategy, automated testing at several levels, configuration management, and deployment strategies that let a release happen without users noticing.

Jenkins is the most widely used open-source automation server for building such pipelines. A modern Jenkins setup describes the pipeline itself in a text file stored with the code (the *Jenkinsfile*) and can describe the Jenkins server too, as code, so that nothing about the delivery process lives only in someone's memory or in settings clicked together in a web interface.

1.2 Problem Statement

The application used in this project is **CivicSense**, a civic complaint platform built with Django and MySQL in Experiment 5 of Technical Training-II and containerised with Docker in the Containerization with Docker course. Citizens file complaints in plain language; a natural-language engine routes them to departments, merges repeat reports and scores their priority for the officers. The application already had a Docker image, a Compose file and a simple GitHub Actions workflow, but releasing a new version still depended on a person:

- **Testing was optional.** Nothing stopped an untested commit from being deployed, and the existing CI lint step ended in `|| true`, so even a syntax error passed.
- **Every release meant downtime.** Replacing the running container stopped the site while the new one migrated the database and started gunicorn.
- **There was no staging step.** A release went straight from the developer's laptop to the only running copy.
- **Nothing undid a bad release.** If a release broke the login, the old version had to be found and redeployed by hand while users were locked out.

- **The CI server was not reproducible.** The Jenkins installation from the lab worksheets had been set up by clicking through its web interface; if the machine died, so did the configuration.

The problem addressed by this project is therefore: **how can every change to CivicSense be built, checked and tested automatically, released first to staging and then to production without any downtime, and rolled back automatically if it misbehaves – using a pipeline and a CI server that are themselves defined entirely as code?**

1.3 Proposed Solution

The project builds a complete delivery pipeline around CivicSense, published in a new repository:

- a **declarative Jenkinsfile** with parallel stages: static analysis and the image build, then unit tests on a real MySQL 8.4 database and a dependency audit, then a smoke test of the image itself;
- a **multibranch** job, so every Git branch is built and tested, while only `main` is deployed – first to **staging**, then to **production**;
- a **blue-green deployment** script: the new release is started next to the running one, checked, and only then does nginx switch over with a graceful reload; a failed smoke test after the switch puts the previous release back by itself;
- **configuration management**: one environment description for staging and production, the differences in small environment files, and secrets generated per environment outside the repository;
- a **Jenkins controller configured as code** (plugins list, Configuration as Code YAML and a start script), and a **GitHub Actions** workflow that checks every pull request independently of Jenkins.

1.4 Scope of the Project

The scope covers version control and the branching workflow, the CI pipeline with its quality gates, continuous deployment to two environments on one machine, rollback, configuration management and the reproducible CI server. Everything ran on the author's laptop: Jenkins 2.580.1, Docker Engine 29.9.0 and Docker Compose 5.6.0 on Ubuntu 26.04.1. All figures in this report – build durations, stage times, test counts, coverage, request counts during releases, rollback times – were measured on 31 October 2026 from real runs of the pipeline and are kept in the Jenkins build records.

The source code, the pipeline, the deployment scripts and this report are published at **https://github.com/iPatan7/civicsense-devops**.

1.5 Organisation of the Report

Chapter 2 explains the DevOps concepts used and lists the objectives. Chapter 3 states the requirements and Chapter 4 the tools. Chapter 5 presents the pipeline, the Jenkins controller, the branching workflow and the blue-green environments. Chapter 6 lists the implementation steps and Chapter 7 explains the main files. Chapter 8 reports every pipeline run and measurement, including the defects found and fixed. Chapter 9 collects the screenshots, and the report closes with the conclusion, the future scope and a step-by-step guide in Appendix A.

CHAPTER 2: BACKGROUND AND PROJECT OBJECTIVES

2.1 DevOps and the Delivery Pipeline

DevOps combines culture, practices and tools so that the people who build software and the people who run it share one goal: frequent, reliable releases. In practice it means automating everything between a commit and production, measuring the result, and feeding problems back to the developers quickly. The pipeline is where this happens. It encodes the release procedure – which checks must pass, in which order, on which environment – so that it is the same for every change and visible to everyone.

2.2 Testing in DevOps and Continuous Testing

Traditional testing happened at the end of a project, in a separate phase. In DevOps the tests run *continuously*: every commit triggers them, and their result decides whether the change may go further. Tests are arranged as a pyramid – many fast unit tests, fewer integration tests, and a few end-to-end checks – and each level runs where it is cheapest. This project uses five levels, all automated:

- **Static analysis** (flake8 and bandit) – finds syntax errors, undefined names and insecure patterns without running the code;
- **Unit and integration tests** – the Django test suite, run inside the production image against a real MySQL 8.4 database, with coverage measurement;
- **Dependency audit** – the exact packages inside the image are checked against public vulnerability databases;
- **Image smoke test** – the image is started on its own to prove it boots and does not run as root;
- **Deployment smoke test** – seven checks through the public address after each release, including a real login.

2.3 Version Control and Branching Strategy

Git is a *distributed* version control system: every clone holds the full history, so developers commit, branch and merge locally and synchronise with a shared repository (here GitHub). A **branching strategy** decides how work flows. This project uses a short-lived feature branch workflow, close to *GitHub flow*: `main` is always releasable; each change is developed on its own branch, checked by the pipeline, reviewed in a pull request and merged into `main`, which is then deployed. Long-lived release branches are avoided because they delay integration and multiply merge conflicts.

2.4 Jenkins

Jenkins is an open-source automation server written in Java. A **controller** stores the configuration, schedules work and serves the web interface; **agents** (or executors on the controller itself) run the builds. Almost every capability comes from **plugins**. A build is triggered manually, by a schedule, by polling the repository (*Poll SCM*), by a webhook from the Git server, or remotely through a URL with a token.

The pipeline is described in a **Jenkinsfile**. The *declarative* syntax used here has a fixed structure – `pipeline`, `agent`, `options`, `stages`, `post` – with `parallel` blocks, `when` conditions and post-conditions such as `always`. A **multibranch pipeline** job scans the repository, creates one sub-job for every branch that contains a Jenkinsfile, and removes the sub-jobs of deleted branches.

2.5 Continuous Integration, Delivery and Deployment

Table 2.1 – Continuous integration, delivery and deployment

Practice	What is automated	In this project
Continuous integration	Build and test every change as soon as it is pushed	Every branch: lint, build, 41 tests on MySQL, audit, image smoke test
Continuous delivery	Every passing build is releasable; release is a decision	Every main build produces a tested, tagged image
Continuous deployment	Every passing build is released automatically	main is deployed to staging and production without a manual step

2.6 Deployment Strategies

How a new version replaces the old one decides whether users notice a release. Table 2.2 compares the common strategies. Blue-green was chosen because it needs no cluster, gives an instant rollback and lets the new version be checked before any user reaches it.

Table 2.2 – Deployment strategies

Strategy	How it works	Trade-off
Recreate	Stop the old version, start the new one	Simple; downtime on every release
Rolling	Replace instances one at a time	No downtime; needs several instances; rollback is another rollout
Blue-green	Start the new version beside the old, switch traffic at once	No downtime, instant rollback; twice the resources during a release
Canary	Send a small share of traffic to the new version first	Lowest risk; needs traffic splitting and metrics

2.7 Configuration Management

Configuration management keeps every environment in a known, reproducible state by describing it in version-controlled files instead of changing servers by hand. Three rules are applied: the same deployment code for every environment, with only a small set of values differing (*configuration as code*); secrets kept out of the repository and generated per environment; and the tools themselves – here the Jenkins controller – configured from files too (*Jenkins Configuration as Code*).

2.8 Project Objectives

The objectives of the project are:

- To build a **declarative Jenkins pipeline** that lints, builds, tests and audits every branch of CivicSense and deploys `main`, with independent stages running in parallel.
- To run the **test suite inside the production image against MySQL 8.4** on every build, and to publish test results, coverage and the exact package list as build records.
- To **deploy without downtime** using a blue-green switch, and to measure how many user requests fail during a release.
- To **roll back automatically** when a release fails its smoke test after the switch, and to measure how long a rollback takes.
- To manage **configuration as code**: one environment description, per-environment files, secrets generated outside Git, and a Jenkins controller rebuilt entirely from files.
- To use a **feature-branch workflow** with checks on every pull request from both Jenkins and GitHub Actions.
- To record the **defects** the pipeline itself exposed, and how they were fixed.

CHAPTER 3: SYSTEM REQUIREMENTS

This chapter lists the resources used. The "used in this project" columns record the actual machine and versions on which every result in Chapter 8 was measured.

3.1 Hardware Requirements

Table 3.1 – Hardware requirements

Component	Minimum	Used in this project
Processor	64-bit dual core	Intel Core i7-10510U, 4 cores / 8 threads
Memory (RAM)	8 GB	15 GiB; Jenkins limited to a 1 GiB heap
Storage	10 GB free	NVMe SSD; Jenkins plugins 63 MB, application image 463 MB
Network	Internet for GitHub and packages	Broadband; Jenkins reaches GitHub over HTTPS

3.2 Software Requirements

Table 3.2 – Software requirements

Software	Version	Purpose
Operating system	Ubuntu 26.04.1 LTS	Build machine and Docker host
Jenkins (WAR)	2.580.1	Automation server running the pipeline
Java (OpenJDK)	25.0.4.1	Runtime of Jenkins
Docker Engine / Compose	29.9.0 / 5.6.0	Image builds, test containers, environments
Git / GitHub CLI	2.53.0 / 2.46.0	Version control, branches, pull requests
Python / Django	3.12.16 / 6.1.2	Application runtime (inside the image)
MySQL / nginx	8.4.13 / 1.27.5	Database and reverse proxy of each environment
flake8 / bandit / pip-audit	7.3.0 / 1.8.6 / 2.9.0	Static analysis and dependency audit (pinned)
coverage / unittest-xml-reporting	latest at build time	Coverage and JUnit XML from the Django tests
GitHub Actions	ubuntu-latest runners	Independent checks on pushes and pull requests
Google Chrome	155	Screenshots of Jenkins and the application

3.3 Functional Requirements

Table 3.3 – Functional requirements of the pipeline

ID	Requirement
FR1	Every push to any branch must be built and tested automatically, without a manual trigger.
FR2	A syntax error, an undefined name, a high-severity security finding or a failing test must stop the build.

ID	Requirement
FR3	The tests must run inside the image that would be deployed, against MySQL, not against SQLite.
FR4	Only `main` may be deployed, and always to staging before production.
FR5	A release must not make any user request fail.
FR6	A release that fails its smoke test must be undone automatically; a manual rollback must also be possible.
FR7	Test results, coverage, analysis reports and the package list of every build must be kept.
FR8	The Jenkins controller, including its job, must be reproducible from files in the repository.

3.4 Non-Functional Requirements

- **Speed of feedback** – a developer should know within a few minutes whether a change is good.
- **Reproducibility** – the same commit must give the same result on every run; tool versions are pinned.
- **Security** – no secrets in Git or in process arguments; no published demo passwords in production; environments reachable only from the machine itself.
- **Transparency** – every stage, its duration and its output visible in Jenkins; every step also runnable by hand.
- **Portability** – the agent needs only Docker and a shell; every tool runs in a container.

CHAPTER 4: TOOLS AND TECHNOLOGIES USED

Table 4.1 lists the tools and the role each plays in the pipeline.

Table 4.1 – Tools and technologies used

Tool	Role	How it is used
Git and GitHub	Version control	Repository, feature branches, pull request #1, the only source Jenkins builds from
Jenkins	Automation server	Multibranch pipeline, two executors, 127.0.0.1:8090
Configuration as Code, Job DSL	Jenkins as code	casc.yaml sets security and creates the multibranch job
Pipeline, Multibranch	Pipelines	Declarative Jenkinsfile, one sub-job per branch
JUnit, Stage View, Pipeline Graph View	Reporting	Test trend, stage durations, graph of each run
Timestamper, ws-cleanup	Housekeeping	Time-stamped logs, workspace clean-up
Docker and Docker Compose	Build and run	Image build, throw-away MySQL for tests, one Compose project per environment
flake8, bandit	Static analysis	Errors and security findings, in a pinned tools image
Django test runner, coverage	Testing	41 tests on MySQL 8.4, JUnit XML, coverage report
pip-audit	Dependency audit	Checks the frozen package list of every image
nginx	Traffic switch	Graceful reload between the blue and green slots
GitHub Actions	Second CI	Tests on a MySQL service container, image build and smoke test on every PR
curl, awk, shell	Smoke tests and measurement	ci/smoke.sh, ci/traffic.sh, ci/measure.sh
Google Chrome	Documentation	Headless screenshots of Jenkins and the portal

4.1 Jenkins and its Plugins

Jenkins runs from the official WAR file as the build user, so builds can use the local Docker engine without root rights. Ten plugins are listed in `jenkins/plugins.txt`; the plugin installation manager resolved them to **77 plugins** with all dependencies (63 MB) in **21 s**. With the plugins in place the controller was fully up **9 s** after it was started, with the job already created from `casc.yaml`.

4.2 Docker as the Build Environment

No language tools are installed on the build machine. flake8, bandit and pip-audit run in a small image with pinned versions (`ci/Dockerfile.tools`); the tests run inside the freshly built application image; MySQL for the tests is a throw-away container with its data in memory (tmpfs). The only requirement for a Jenkins agent is therefore Docker and a POSIX shell, which makes the pipeline portable to any agent.

4.3 GitHub Actions

GitHub Actions runs the same tests on GitHub's hosted runners for every push to `main` and every pull request. It is deliberately independent of Jenkins: a pull request shows a green or red check on GitHub even when the Jenkins machine is switched off, and the two systems confirm each other's results.

CHAPTER 5: SYSTEM DESIGN

5.1 Pipeline Architecture

Figure 5.1 shows the complete flow. A push reaches GitHub; GitHub Actions checks pushes to `main` and pull requests; Jenkins scans the repository every minute and builds every branch that changed. Inside Jenkins, independent work runs in parallel: static analysis beside the image build, and the unit tests beside the dependency audit. The deployment stages run only for `main`.

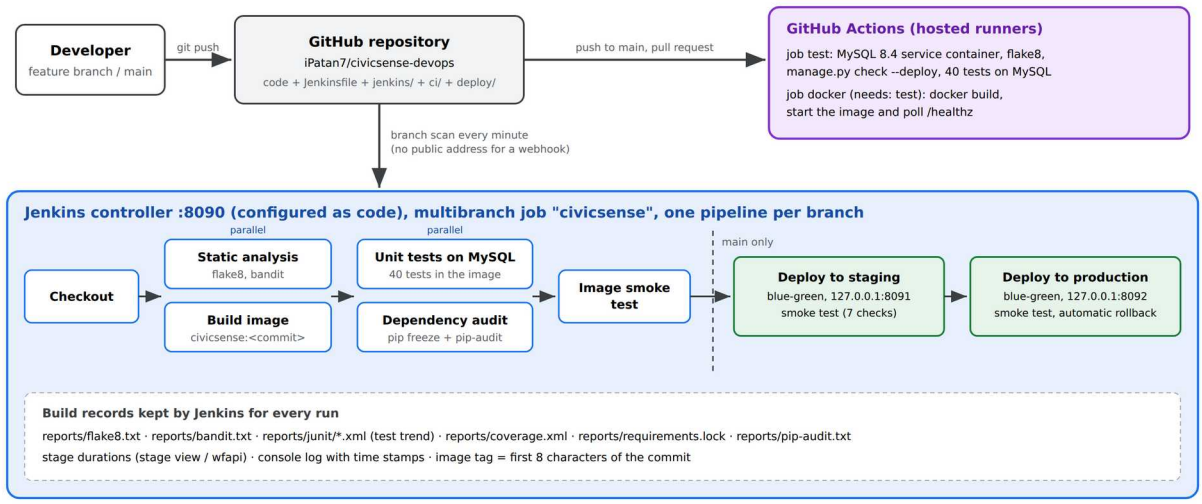


Figure 5.1 – The delivery pipeline: GitHub Actions and the Jenkins multibranch pipeline

Table 5.1 – Quality gates of the pipeline

Gate	Tool	Effect on the build
Code errors	flake8 E9, F63, F7, F82	Fails the build
Code style	flake8 (max line length 100)	Reported only (reports/flake8.txt)
Security patterns	bandit	High severity and confidence fails; others reported
Tests	Django tests on MySQL 8.4	Any failure fails the build; JUnit trend kept
Known vulnerabilities	pip-audit on the image's packages	Stage marked unstable; release continues
Image boots, not root	ci/smoke-image.sh	Fails the build
Release works for users	ci/smoke.sh (7 checks)	Fails the build; production is rolled back

5.2 Jenkins Configured as Code

The controller is built from three files (Figure 5.2). `plugins.txt` lists the plugins; `casc.yaml` defines the security realm with one administrator, read-only access for anonymous visitors, the system message, the URL, timestamps for all pipelines, and – through the Job DSL plugin – the multibranch job itself; `run-jenkins.sh` installs the plugins and starts Jenkins with the

setup wizard disabled. The administrator password is the only input that is not in Git; it comes from an environment variable.

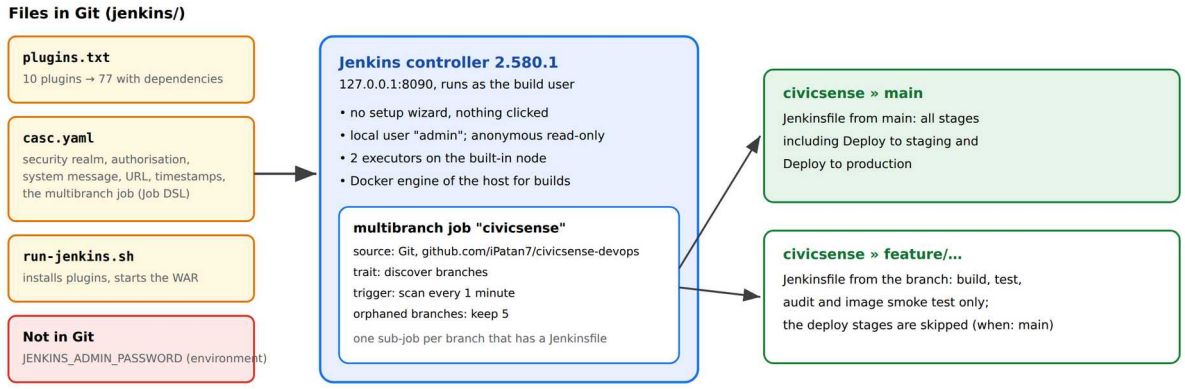


Figure 5.2 – Jenkins rebuilt from files: plugins, configuration and the job

Because GitHub cannot reach a laptop without a public address, a webhook was not possible. The multibranch job therefore scans the repository every minute, the multibranch equivalent of Poll SCM from the lab worksheets. The measured delay between a push and the start of its build was between 17 and 69 s.

5.3 Branching Workflow

Figure 5.3 shows the workflow. A change starts on a feature branch. Its Jenkins sub-job runs all checks but skips the deploy stages, and the pull request into `main` runs GitHub Actions as well. Only after both are green is the branch merged; the merge commit is then built on `main` and deployed.

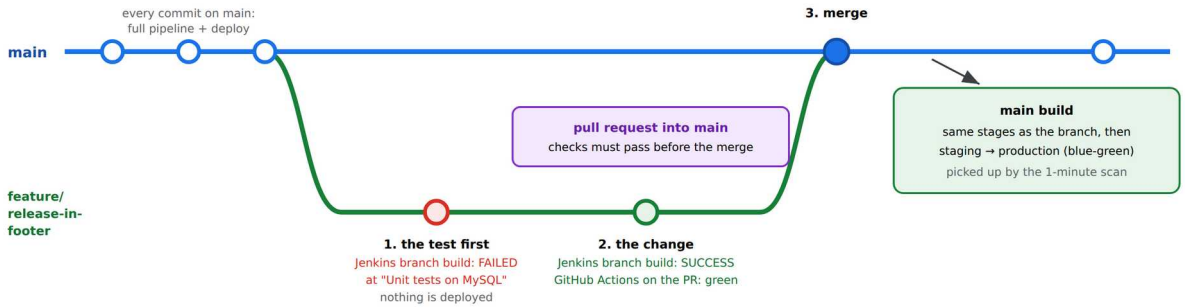


Figure 5.3 – Feature branch, pull request and merge into main

5.4 Environments and Blue-Green Deployment

Each environment – staging on port 8091 and production on port 8092 – is one Docker Compose project with four services (Figure 5.4): MySQL, two web slots (`web-blue` and `web-green`) and nginx. nginx sends traffic to the slot named in a small file, `live.conf`. A release goes to the idle slot, is checked there while users are still served by the live slot, and only then is `live.conf` rewritten and nginx reloaded. A reload starts new worker processes with the new

configuration while the old workers finish the requests they are handling, so no connection is cut.

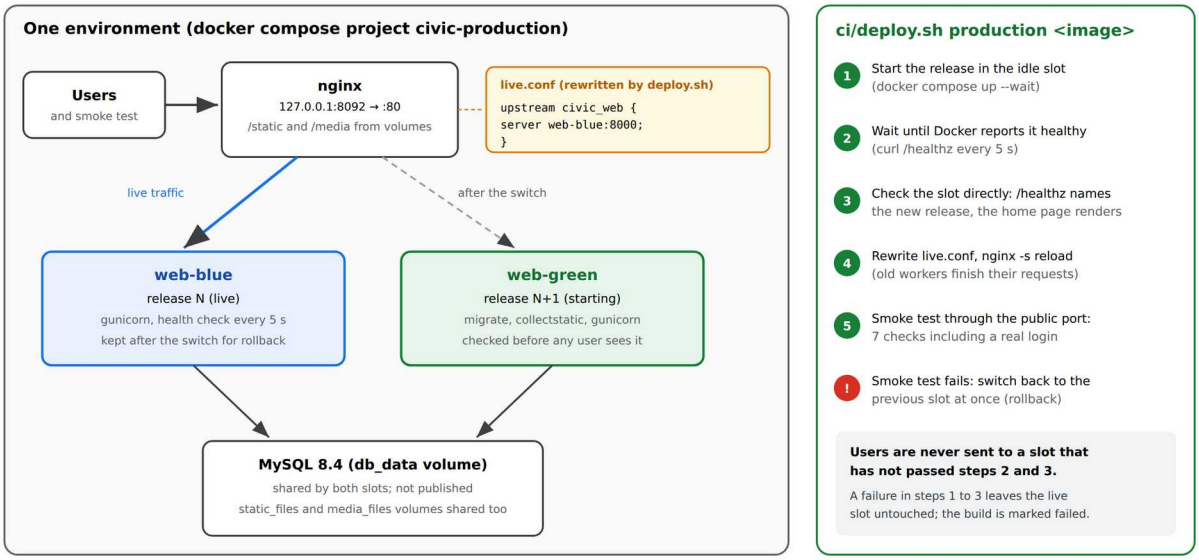


Figure 5.4 – One environment: two web slots behind nginx, and the steps of a release

The previous slot keeps running after the switch. A rollback is therefore not a redeployment but one more switch, and it takes well under a second. Both slots share the database and the static and media volumes, so a release must keep the database schema compatible with the version it replaces – the usual constraint of blue-green deployment.

5.5 Configuration Management Design

Table 5.2 – Where each kind of configuration lives

Kind	Location	Example
Deployment description	deploy/compose.yml (one file)	Services, volumes, health checks
Per-environment values	deploy/env/staging.env, production.env	Port, workers, log level, demo data on/off
Secrets	~/.civicsense/<env>/secrets.env (generated, mode 600)	Django secret key, database passwords, smoke-test password
Environment state	~/.civicsense/<env>/nginx/live.conf, slot-*, history.log	Which slot is live, what each slot runs
Pipeline	Jenkinsfile	Stages, conditions, records
CI server	jenkins/plugins.txt, jenkins/casc.yaml	Plugins, security, the job

5.6 Security of the Pipeline

Table 5.3 – Security measures in the pipeline

Measure	Implementation
No secrets in Git	Passwords generated with openssl on the first deployment of each environment; Jenkins password from the environment

Measure	Implementation
No secrets in process lists	The smoke-test password is passed to `docker compose exec` by name and to curl on standard input
No demo passwords in production	Demo data only where the environment file sets LOAD_DEMO_DATA=1 (staging), never with --reset
Dedicated smoke-test account	A citizen account per environment whose password exists only in that environment's secrets file
Local-only exposure	Jenkins and both nginx instances listen on 127.0.0.1
Least privilege for builds	Jenkins runs as an ordinary user; containers run as uid 1000, checked in every build
Read-only by default	Anonymous visitors can read Jenkins but not start or change anything
Supply chain	Pinned tool versions; every image's package list recorded and audited

CHAPTER 6: IMPLEMENTATION STEPS

The project was implemented in the order below. Every step produced files in the repository, and every pipeline stage calls a script in `ci/`, so each step can also be run by hand.

6.1 Step 1: Start the Repository from the Application History

The new repository `civicsense-devops` was created from the history of the Docker project, so the application, its Dockerfile and its first CI workflow keep their commits. The Docker report was removed from the new repository, and the README was rewritten for the DevOps project.

6.2 Step 2: Make the Release Visible

`/healthz` was extended to return the release (the commit the image was built from) and the blue-green slot. A smoke test can then prove from outside which build is answering. A unit test was added for it, bringing the suite to 40 tests.

6.3 Step 3: Static Analysis and the Tools Image

`ci/Dockerfile.tools` builds an image with pinned versions of flake8 7.3.0, bandit 1.8.6 and pip-audit 2.9.0. `ci/lint.sh` writes full reports to `reports/` and fails only on real errors and high-severity findings.

6.4 Step 4: Tests Inside the Image, on MySQL

`ci/test.sh` starts a MySQL 8.4 container on a private network with its data in tmpfs, waits until it accepts TCP connections, and runs the Django tests inside the image that was just built, with the XML test runner and coverage. JUnit files and the coverage report are written to `reports/` through a bind mount.

6.5 Step 5: Dependency Audit and Image Smoke Test

`ci/audit.sh` freezes the package list of the image into `reports/requirements.lock` and checks it with pip-audit. `ci/smoke-image.sh` starts the image on its own with production settings, waits for `/healthz` and checks that the process is not root.

6.6 Step 6: The Environments and the Blue-Green Script

`deploy/compose.yml` describes one environment, and `deploy/env/*.env` holds the differences. `ci/deploy.sh <env> <image>` performs a release; `ci/deploy.sh <env> rollback` and `status` operate the environment.

6.7 Step 7: The Deployment Smoke Test

`ci/smoke.sh` runs seven checks against the public address: the health endpoint names the expected release, the home page renders, a stylesheet is served by nginx, the login page carries a CSRF token, a citizen can log in, sees the own complaints page, and is refused by the authority dashboard.

6.8 Step 8: The Jenkinsfile

The declarative Jenkinsfile ties the scripts together: checkout with the commit as image tag and build name, two parallel groups, the image smoke test, and the two deploy stages limited to `main` with `when { branch 'main' }`. A `post { always }` block archives `reports/` and publishes the JUnit results even when a stage fails.

6.9 Step 9: Jenkins as Code

`jenkins/plugins.txt`, `jenkins/casc.yaml` and `jenkins/run-jenkins.sh` were written, and a fresh controller was started from them on port 8090. The multibranch job appeared without any click and found the `main` branch on its first scan.

```
JENKINS_ADMIN_PASSWORD='…' jenkins/run-jenkins.sh      # Jenkins on
http://localhost:8090
```

6.10 Step 10: GitHub Actions and the Branching Workflow

The GitHub Actions lint step was changed to fail on the same errors as Jenkins. The footer feature was then developed on `feature/release-in-footer`: the test first, then the change, a pull request with checks from both systems, and a merge into `main`.

CHAPTER 7: IMPLEMENTATION AND CODE EXPLANATION

7.1 Project Structure

Repository layout (github.com/iPatan7/civicsense-devops)

```
civicsense-devops/
├── Jenkinsfile                  the declarative pipeline
├── jenkins/
│   ├── plugins.txt              10 plugins (77 with dependencies)
│   ├── casc.yaml                security, settings and the multibranch job
│   └── run-jenkins.sh           builds and starts the controller
├── ci/
│   ├── Dockerfile.tools         pinned flake8, bandit, pip-audit
│   ├── lint.sh, test.sh         static analysis; tests on MySQL
│   ├── audit.sh                 pip-audit of the image's packages
│   ├── smoke-image.sh           the image boots and is not root
│   ├── deploy.sh                blue-green release, rollback, status
│   ├── smoke.sh                 7 checks through the public address
│   ├── traffic.sh, measure.sh   requests during a release
│   └── cleanup.sh               CI containers and old image tags
├── deploy/
│   ├── compose.yml              db + web-blue + web-green + nginx
│   ├── nginx.conf               proxy; upstream comes from live.conf
│   └── env/                     staging.env, production.env
├── .github/workflows/ci.yml     GitHub Actions
├── Dockerfile, docker/          production image and entrypoint
└── civicsense/ accounts/ …      the Django application
```

7.2 The Jenkinsfile

Jenkinsfile

```
// CivicSense delivery pipeline (declarative).
//
// Every branch:  static analysis and the image build run in parallel, then the
//                unit tests on MySQL and a dependency audit in parallel, then a
//                smoke test of the image on its own.
// main only:     blue-green deployment to staging and then to production. If the
//                smoke test fails after the production switch, ci/deploy.sh puts
//                the previous release back by itself.
//
// The stages only call scripts in ci/, so every step can also be run by hand.
pipeline {
    agent any


    options {
        skipDefaultCheckout(true)
        timestamps()
        timeout(time: 30, unit: 'MINUTES')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '30'))
    }


    stages {
        stage('Checkout') {
            steps {
                script {
                    def scmVars = checkout scm
                    env.GIT_COMMIT = scmVars.GIT_COMMIT
                    env.GIT_SHORT = env.GIT_COMMIT.take(8)
                    env.IMAGE = "civicsense:${env.GIT_SHORT}"
                    currentBuild.displayName = "#${env.BUILD_NUMBER} ${env.GIT_SHORT}"
                    currentBuild.description = sh(script: 'git log -1 --format=%s',
returnStdout: true).trim()
                }
                sh 'rm -rf reports && mkdir -p reports'
            }
        }
```

```
        stage('Build and analyse') {
            parallel {
                stage('Static analysis') {
                    steps { sh 'ci/lint.sh' }
                }
                stage('Build image') {
                    steps {
                        sh 'docker build -t "$IMAGE" --label
"org.opencontainers.image.revision=$GIT_COMMIT" .'
                    }
                }
            }
        }

        stage('Test') {
            parallel {
                stage('Unit tests on MySQL') {
                    steps { sh 'ci/test.sh "$IMAGE"' }
                    post {
                        always { junit allowEmptyResults: true, testResults:
'reports/junit/*.xml' }
                    }
                }
                stage('Dependency audit') {
                    steps {
                        // A known vulnerability marks the stage, but does not stop the
release.
                        catchError(buildResult: 'SUCCESS', stageResult: 'UNSTABLE') {
                            sh 'ci/audit.sh "$IMAGE"'
                        }
                    }
                }
            }
        }

        stage('Image smoke test') {
            steps { sh 'ci/smoke-image.sh "$IMAGE"' }
        }

        stage('Deploy to staging') {
            when { branch 'main' }
            steps { sh 'ci/deploy.sh staging "$IMAGE"' }
        }

        stage('Deploy to production') {
            when { branch 'main' }
            steps { sh 'ci/deploy.sh production "$IMAGE"' }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'reports/**', allowEmptyArchive: true
            sh 'ci/cleanup.sh'
        }
    }
}
```

`skipDefaultCheckout` and an explicit `checkout scm` let the pipeline take the commit hash and use its first eight characters as the image tag, so every image can be traced to its commit. `disableConcurrentBuilds` makes builds of the same branch queue instead of running over each other, which matters for the deploy stages. The `catchError` around the audit marks only that stage when a vulnerability is found, as decided in the quality gates.

7.3 Jenkins Configuration as Code

jenkins/casc.yaml

```
# Jenkins Configuration as Code for the CivicSense controller.
#
# Nothing is configured by clicking in the web interface: security, the system
# settings and the pipeline job all come from this file, so the controller can be
# thrown away and rebuilt identically with jenkins/run-jenkins.sh.
jenkins:
  systemMessage: "CivicSense CI/CD. This controller is configured from jenkins/casc.yaml."
  numExecutors: 2
  mode: NORMAL
  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: "admin"
          name: "CI administrator"
          password: "${JENKINS_ADMIN_PASSWORD}"   # taken from the environment, never
stored in Git
  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: true                    # dashboards are readable, changes need a
login
  markupFormatter: "plainText"

unclassified:
  location:
    url: "http://localhost:8090/"
  timestamper:
    allPipelines: true

jobs:
  - script: |
      multibranchPipelineJob('civicsense') {
        displayName('CivicSense')
        description('Builds and tests every branch of civicsense-devops. Only main is
deployed: staging first, then production (blue-green).')
        branchSources {
          branchSource {
            source {
              git {
                id('civicsense-devops')
                remote('https://github.com/iPatan7/civicsense-devops.git')
                traits {
                  gitBranchDiscovery()
                }
              }
            }
          }
        }
        // There is no public address for a GitHub webhook to reach, so the
        // repository is scanned for new commits every minute instead.
        triggers {
          periodicFolderTrigger {
            interval('1m')
          }
        }
        orphanedItemStrategy {
          discardOldItems {
            numToKeep(5)
          }
        }
      }
```

`${JENKINS_ADMIN_PASSWORD}` is resolved from the environment when Jenkins starts. The `jobs` section is Job DSL: it creates the multibranch job with a Git branch source, branch discovery and the one-minute scan trigger.

7.4 The Blue-Green Deployment Script

ci/deploy.sh (the release part)

```
IMAGE=$TARGET RELEASE=${TARGET#*:}
NEW=blue; [ "$LIVE" = blue ] && NEW=green
SLOT=$(echo "$NEW" | tr a-z A-Z)
export "WEB_${SLOT}_IMAGE=$IMAGE" "WEB_${SLOT}_RELEASE=$RELEASE"
cp "$ROOT/deploy/nginx.conf" "$STATE_DIR/nginx/default.conf"
log "$ENV_NAME: live slot ${LIVE:-none}, starting $IMAGE in slot $NEW"
T0=$(now)


dc up -d --wait db >/dev/null 2>&1
t=$(now)
if ! dc up -d --no-deps --wait --wait-timeout 240 "web-$NEW" >/dev/null 2>&1; then
  log "slot $NEW did not become healthy; live slot ${LIVE:-none} untouched"
  dc logs --tail 30 "web-$NEW"; record "FAILED start $IMAGE in $NEW"; exit 1
fi
log "slot $NEW healthy after $(since "$t") s (migrations and static files included)"


# Check the new slot directly, before any user is sent to it.
dc exec -T "web-$NEW" curl -fsS -H "Host: localhost" http://localhost:8000/healthz | grep -q
"\"release\": \"$RELEASE\"" \
  && dc exec -T "web-$NEW" curl -fsS -o /dev/null -H "Host: localhost" http://localhost:8000/ \
  || { log "slot $NEW answers wrongly; live slot ${LIVE:-none} untouched"; record "FAILED check
$IMAGE in $NEW"; exit 1; }
log "slot $NEW serves release $RELEASE"


# The smoke test logs in with its own citizen account, whose password exists only
# in this environment's secrets file (no shared or published password).
export SMOKE_PASSWORD             # passed by name, so the value never appears in a process list
dc exec -T -e SMOKE_PASSWORD "web-$NEW" python manage.py shell -c '
import os
from django.contrib.auth.models import User
user, _ = User.objects.get_or_create(username="smoke-test", defaults={"email":
"smoke@localhost"})
user.set_password(os.environ["SMOKE_PASSWORD"])
user.save()' >/dev/null


# Demonstration data (with its well-known demo passwords) only where the
# environment file asks for it, and only into an empty database: never reset.
if [ "${LOAD_DEMO_DATA:-0}" = 1 ] && [ "$(dc exec -T "web-$NEW" python manage.py shell -c 'from
complaints.models import Complaint; print(Complaint.objects.count())' 2>/dev/null | tail -1)" = 0
]; then
  log "empty database: loading the demonstration data"
  dc exec -T "web-$NEW" python manage.py seed_demo >/dev/null
fi


t=$(now); switch_to "$NEW" || true
log "nginx switched ${LIVE:-none} -> $NEW in $(since "$t") s"


if ! SMOKE_USER=smoke-test "$ROOT/ci/smoke.sh" "http://localhost:$PUBLIC_PORT" "$RELEASE"; then
  if [ -n "$LIVE" ] && healthy "$LIVE"; then
    t=$(now); switch_to "$LIVE"
    log "smoke test failed: rolled back to slot $LIVE ($(cat "$STATE_DIR/slot-$LIVE")) in $(since
"$t") s"
    record "ROLLED BACK $IMAGE in $NEW -> $LIVE"
    echo "$IMAGE (failed)" > "$STATE_DIR/slot-$NEW"
  else
    log "smoke test failed and there is no previous release to go back to"
    record "FAILED smoke $IMAGE in $NEW"
  fi
  exit 1
fi


echo "$IMAGE" > "$STATE_DIR/slot-$NEW"
record "deployed $IMAGE in $NEW"
log "$ENV_NAME now serves $RELEASE from slot $NEW (total $(since "$T0") s)${LIVE:+; slot $LIVE
kept for rollback}"
```

Steps that fail before the switch leave the live slot untouched and stop the build. The switch waits until the public address really answers from the new slot, because `nginx -s reload` returns before the new worker processes have taken over (Section 8.12). If the smoke test then

fails, the script switches back at once and marks the slot as holding a failed release, so a later manual rollback cannot select it by mistake.

7.5 The Environment Description

deploy/compose.yml

```
# One CivicSense environment (staging or production), deployed blue-green.
#
# Two web slots run the application side by side: one is live, the other is idle
# or holds the previous release. nginx sends traffic to whichever slot is named in
# $STATE_DIR/nginx/live.conf; ci/deploy.sh rewrites that file and reloads nginx.
#
# Used only through ci/deploy.sh, which supplies the project name, the environment
# file (deploy/env/<env>.env), the secrets file and the image of each slot.


x-web: &web
  restart: unless-stopped
  command: ["gunicorn", "civicsense.wsgi:application", "--bind", "0.0.0.0:8000",
            "--workers", "${GUNICORN_WORKERS:-2}", "--timeout", "60", "--access-logfile", "-"]
  volumes:
    - static_files:/app/staticfiles
    - media_files:/app/media
  depends_on:
    db:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "curl", "-fsS", "http://localhost:8000/healthz"]
    interval: 5s
    timeout: 3s
    retries: 3
    start_period: 120s


x-web-env: &web-env
  DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY}
  DJANGO_DEBUG: "0"
  DJANGO_ALLOWED_HOSTS: ${DJANGO_ALLOWED_HOSTS}
  DJANGO_CSRF_TRUSTED_ORIGINS: ${DJANGO_CSRF_TRUSTED_ORIGINS}
  DJANGO_LOG_LEVEL: ${DJANGO_LOG_LEVEL:-INFO}
  DB_ENGINE: mysql
  DB_NAME: civicsense
  DB_USER: civic
  DB_PASSWORD: ${DB_PASSWORD}
  DB_HOST: db
  DB_PORT: "3306"
  SEED_DEMO_DATA: "0"


services:
  db:
    image: mysql:8.4
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: civicsense
      MYSQL_USER: civic
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -uroot -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 30


  web-blue:
    <<: *web
    image: ${WEB_BLUE_IMAGE:-civicsense:none}
    environment:
      <<: *web-env
      CIVIC_RELEASE: ${WEB_BLUE_RELEASE:-none}
      CIVIC_SLOT: blue


  web-green:
    <<: *web
    image: ${WEB_GREEN_IMAGE:-civicsense:none}
    environment:
```

```
      <<: *web-env
      CIVIC_RELEASE: ${WEB_GREEN_RELEASE:-none}
      CIVIC_SLOT: green


  nginx:
    image: nginx:1.27-alpine
    restart: unless-stopped
    ports:
      - "127.0.0.1:${PUBLIC_PORT}:80"                # this machine only
    volumes:
      - ${STATE_DIR}/nginx:/etc/nginx/conf.d:ro      # default.conf + live.conf
      - static_files:/app/staticfiles:ro
      - media_files:/app/media:ro


volumes:
  db_data:
  static_files:
  media_files:
```

The two web slots share one definition through YAML anchors and differ only in their image and the release and slot names. The health check runs every five seconds, so `docker compose up --wait` returns as soon as a slot is ready. Every value that differs between environments is a variable, filled from the environment file:

deploy/env/production.env

```
# Production: secrets are generated on the first deployment in
# ~/.civicsense/production/secrets.env and never committed.
ENV_NAME=production
PROJECT=civic-production
PUBLIC_PORT=8092
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
DJANGO_CSRF_TRUSTED_ORIGINS=http://localhost:8092,http://127.0.0.1:8092
DJANGO_LOG_LEVEL=WARNING
GUNICORN_WORKERS=3
# No demonstration data (and so no demo passwords) in production.
LOAD_DEMO_DATA=0
```

7.6 The Smoke Test

ci/smoke.sh (the checks)

```
health() {
  get 200 /healthz && has '"status": "ok"' || return 1
  [ -z "$EXPECT" ] || has "\"release\": \"$EXPECT\""
}
home() { get 200 / && has "CivicSense"; }
static() {
  css=$(curl -s "$URL/" | grep -o '/static/[^"]*\.css' | head -1)
  [ -n "$css" ] || { echo "no stylesheet linked from the home page"; return 1; }
  get 200 "$css"
}
login_page() { get 200 /accounts/login/ && has csrfmiddlewaretoken; }
login() {
  token=$(awk '$6 == "csrftoken" { print $7 }' "$JAR")
  # the password goes to curl on stdin, not on its command line
  got=$(printf '%s' "$SMOKE_PASSWORD" | curl -s -o /dev/null -w '%{http_code} %{redirect_url}' -b
"$JAR" -c "$JAR" \
    -H "Referer: $URL/accounts/login/" -H "Origin: $URL" \
    --data-urlencode "csrfmiddlewaretoken=$token" -d "username=$SMOKE_USER" --data-urlencode
"password@-" \
    "$URL/accounts/login/")
  case "$got" in 302*) ;; *) echo "login returned $got (expected a 302 redirect)"; return 1 ;;
esac
}
my_complaints() { get 200 /mine/ && has 'accounts/logout'; }
access_control() {
  got=$(curl -s -o /dev/null -w '%{http_code}' -b "$JAR" "$URL/authority/")
  case "$got" in 302|403) ;; *) echo "a citizen got $got from the authority dashboard"; return
```

```
1 ;; esac
}

echo "smoke test of $URL${EXPECT:+ (expecting release $EXPECT)}"
check "health endpoint and release"        health
check "home page renders"                  home
check "static files served by nginx"       static
check "login page with CSRF token"         login_page
check "citizen can log in"                 login
check "citizen sees own complaints"        my_complaints
check "authority dashboard is protected"   access_control
echo "smoke test passed"
```

7.7 Tests on MySQL Inside the Image

ci/test.sh

```
set -eu
IMAGE=$1
ID=$(printf '%s' "${BUILD_TAG:-local-$$}" | tr -c 'a-zA-Z0-9_.-' '-' | tr 'A-Z' 'a-z')
NET="ci-$ID" DB="ci-$ID-db"
mkdir -p reports/junit
rm -f reports/junit/*.xml


cleanup() { docker rm -f "$DB" >/dev/null 2>&1 || true; docker network rm "$NET" >/dev/null 2>&1
|| true; }
trap cleanup EXIT
docker network create "$NET" >/dev/null
docker run -d --name "$DB" --network "$NET" --tmpfs /var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=ci-root -e MYSQL_DATABASE=civicsense mysql:8.4 \
  --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci >/dev/null


# A TCP ping only succeeds once MySQL has finished initialising (the temporary
# server it starts first does not listen on the network).
i=0
until docker exec "$DB" mysqladmin ping -h 127.0.0.1 -uroot -pci-root --silent >/dev/null 2>&1;
do
  i=$((i + 1)); [ $i -lt 120 ] || { docker logs "$DB" | tail -20; exit 1; }; sleep 1
done
echo "MySQL 8.4 ready after ${i} s"


docker run --rm --network "$NET" -v "$PWD/reports:/app/reports" \
  -e DB_ENGINE=mysql -e DB_HOST="$DB" -e DB_PORT=3306 -e DB_NAME=civicsense \
  -e DB_USER=root -e DB_PASSWORD=ci-root -e DJANGO_SECRET_KEY=ci \
  -e TEST_OUTPUT_DIR=/app/reports/junit \
  --entrypoint sh "$IMAGE" -c '
    pip install --user --quiet --disable-pip-version-check unittest-xml-reporting coverage &&
    python -m coverage run --source=accounts,complaints,dashboard,nlpengine \
        --omit="*/migrations/*,*/tests.py" manage.py test --noinput \
        --testrunner xmlrunner.extra.djangotestrunner.XMLTestRunner &&
    python -m coverage report > reports/coverage.txt &&
    python -m coverage xml -o reports/coverage.xml &&
    echo "coverage: $(tail -1 reports/coverage.txt)"'
```

The test database is created by the root account of the throw-away MySQL, which is acceptable because the container exists only for this build and is removed by the `trap` even when the tests fail.

CHAPTER 8: TESTING AND RESULTS

8.1 Testing Strategy

The pipeline was tested by using it. Between 12:25 and 14:18 on 31 October 2026 the pipeline ran eleven times, each time for a real commit: fixes, a feature developed on a branch and merged through a pull request, a security improvement, one deliberately faulty configuration change, and a fix for a problem first seen in a screenshot of production. Every change was built by Jenkins exactly as any later change will be. The results below come from the Jenkins build records (stage durations from the pipeline REST API, console logs, JUnit files and archived reports), from GitHub Actions, and from two measuring scripts:

- `ci/traffic.sh` sends a request to the production home page and one to a stylesheet every 0.1 s and records every status code. It ran for 330 s around every production release from build #3 to build #8, so each of these releases is covered from before the push until after the switch.
- `ci/measure.sh` runs a deployment or a rollback by hand while `ci/traffic.sh` is running, and reports both.

8.2 Pipeline Runs

Table 8.1 lists every run. The two failures on `main` and the failure on the feature branch are part of the results: each was caught by the pipeline before it could affect users.

Table 8.1 – All pipeline runs

Run	Change	Result	Time	What happened
main #1	Pipeline and Jenkins as code (f1d99490)	FAILURE	39.1 s	Scripts not executable on the Jenkins checkout (Section 8.12)
main #2	Mark the CI scripts executable	SUCCESS	197.5 s	First deployment of staging and production
main #3	GitHub Actions lint fails on real errors	SUCCESS	173.2 s	Release under traffic: 1 static file request failed
main #4	Keep static files during a release	SUCCESS	168.6 s	Release under traffic: 0 failed
feature #1	Footer test only	FAILURE	152.0 s	1 of 41 tests failed; nothing deployed
feature #2	Footer test and change	SUCCESS	137.8 s	41 of 41 tests passed; deploy stages skipped
main #5	Merge of pull request #1	SUCCESS	167.3 s	Footer released; 0 failed requests
main #6	Tracking ids from the secrets module	SUCCESS	223.2 s	bandit findings 25 → 24; 0 failed requests
main #7	Faulty production configuration	FAILURE	205.9 s	Smoke test failed; production rolled back automatically
main #8	Revert of the faulty configuration	SUCCESS	168.7 s	Production healthy again; 0 failed requests

Run	Change	Result	Time	What happened
main #9	Readable release in the footer	SUCCESS	191.3 s	Fix for a defect seen in a screenshot

8.3 Stage Durations

The first successful run (#2) shows the cost of a first deployment: staging took 47.2 s because it created its database and loaded the demonstration data, production 25.8 s. Table 8.2 summarises the six normal releases after that (#3, #4, #5, #6, #8 and #9).

Table 8.2 – Stage durations of the six normal releases

Stage	Minimum	Mean	Maximum
Static analysis	7.3 s	8.0 s	8.8 s
Build image (cached dependencies)	4.1 s	4.4 s	4.7 s
Unit tests on MySQL	123.8 s	139.0 s	181.5 s
Dependency audit	6.2 s	6.9 s	7.9 s
Image smoke test	5.5 s	5.6 s	5.8 s
Deploy to staging	11.8 s	12.2 s	13.2 s
Deploy to production	10.9 s	11.2 s	11.8 s
Whole pipeline	167.3 s	182.0 s	223.2 s

Two observations follow. First, the parallel groups work: static analysis and the dependency audit add almost nothing to the total because they finish long before their partners (the image build and the tests). Second, the tests dominate: about three quarters of every run is the test stage. Of that stage, 7–8 s is MySQL starting in tmpfs and 89–103 s the tests themselves; the rest – installing the two test tools with pip inside the image and writing the reports – varied between 15 and 75 s from run to run, which explains most of the spread in the total. The image build took only about 4 s because Docker reused the dependency layers and rebuilt only the layer with the source code. A deployment of one environment took about 12 s, of which the new slot needed about 6 s to migrate, collect the static files and pass its first health check, and the nginx switch 0.4–0.6 s.



Figure 8.1 – Jenkins stage view of the main branch: every stage of every run with its duration

8.4 Static Analysis and Dependency Audit

flake8 found **no** errors and **no** style warnings at a line length of 100. bandit found **25** findings of low severity, all of the same kind (B311, use of the `random` module): 24 in the command that generates demonstration data, where predictability does not matter, and one in `generate_tracking_id`, where it does – a tracking id is all it takes to open a complaint. That one was fixed in build #6 by using the `secrets` module, and the count dropped to **24**. pip-audit checked the **7** Python packages frozen from every image (Django 6.1.2, gunicorn 26.2.0, mysqlclient 2.3.0, whitenoise 6.12.0, asgiref, sqlparse and python-dotenv) and found **no known vulnerabilities** (Figure 8.2).

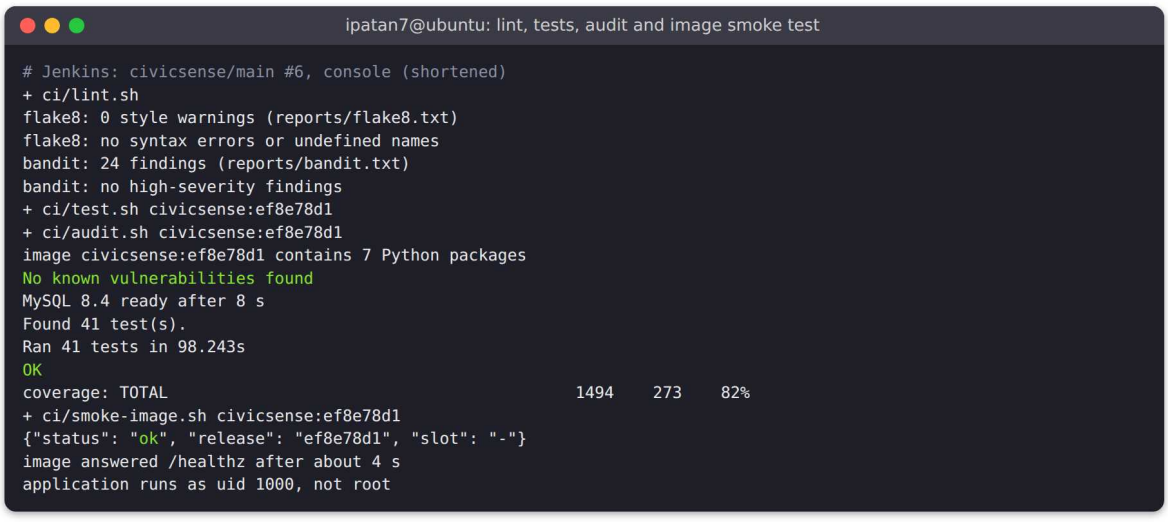


Figure 8.2 – Build #6: static analysis, 41 tests on MySQL 8.4, dependency audit and image smoke test

8.5 Unit Tests and Coverage

The suite grew from 39 tests (inherited from the Docker project) to 40 with the health endpoint test and to **41** with the footer test. In every successful run all tests passed inside the production image against MySQL 8.4. Statement coverage is **82 %** (1,221 of 1,494 statements). The report also shows where tests are missing: 121 of the 273 uncovered statements belong to the demonstration-data command, which no test runs; without it, coverage of the application code is 89 %. The weakest application modules are the account views (65 %) and the citizen views (70 %).

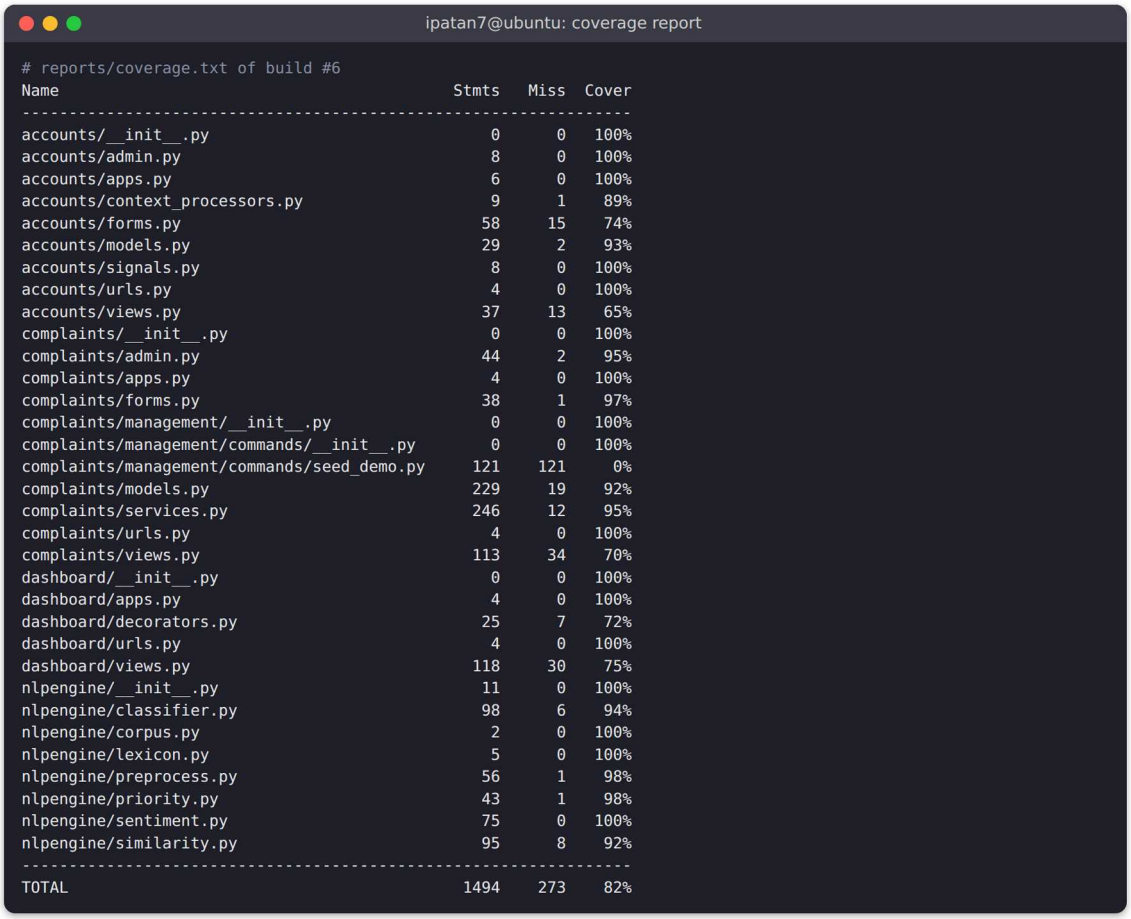


Figure 8.3 – Coverage report archived by build #6

8.6 Branching Workflow and a Test-First Change

The footer feature was developed on `feature/release-in-footer`. The first commit contained only the new test. Jenkins created a sub-job for the branch on its next scan and the build failed in the test stage – **40 passed, 1 failed** – with the message that the expected text was not in the page; the deploy stages did not run. GitHub Actions came to the same result on the pull request: its `test` job failed after 94 s and its `docker` job was skipped. The second commit added a context processor and one line in the footer template; both systems turned green

(Jenkins 41 of 41 tests, GitHub Actions `test` 71 s and `docker` 40 s), the pull request was merged, and Jenkins built and released the merge commit as build #5 (Figure 8.4).

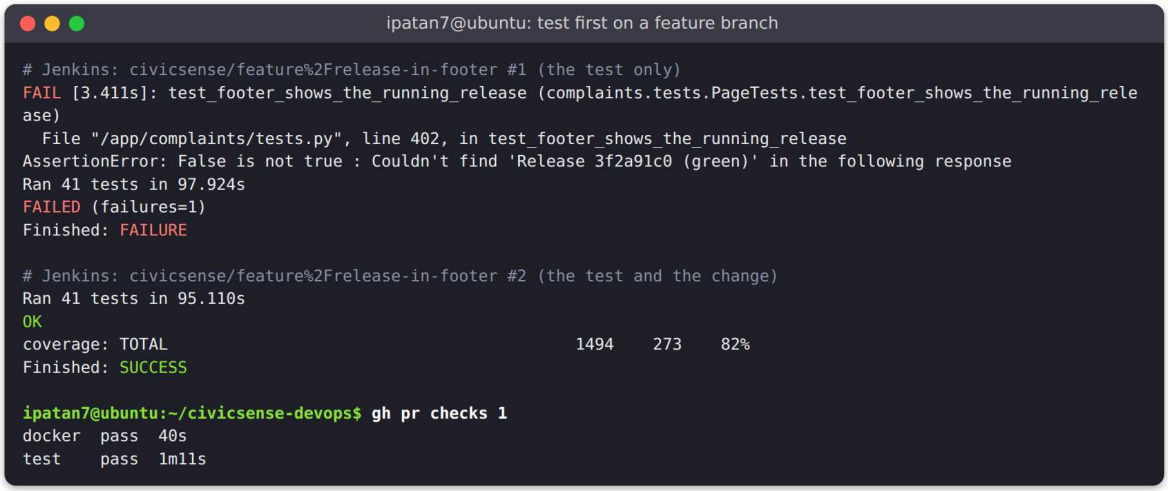


Figure 8.4 – The test first (red in Jenkins), then the change (green in Jenkins and on the pull request)

Table 8.3 – GitHub Actions runs

Commit	Event	Result	Duration
054a014	push to main	success	135 s
f1d9949	push to main	success	121 s
55779fd	push to main	success	114 s
0b7bd0a	push to main	success	123 s
50f6f38	push to main	success	123 s
221f731	pull request	failure	94 s (the test-first commit)
aefc08d	pull request	success	118 s
b9b6e55	push to main	success	129 s (merge of pull request #1)
ef8e78d	push to main	success	125 s
d062da3	push to main	success	161 s (the faulty configuration is invisible to tests)
a24d61d	push to main	success	112 s

8.7 Zero-Downtime Releases

Every production release from build #3 on was made while `ci/traffic.sh` was sending requests to production (Figure 8.6). Table 8.4 shows the result. In build #3 exactly **one** request failed: a stylesheet answered *404* while the new slot was starting. The cause was `collectstatic --clear` in the container entrypoint, which deletes all static files before copying them again – harmless with one container, but in a blue-green release the new slot shares the static volume with the live one, so for a moment the live slot's stylesheet did not exist. The same 404 had already appeared once (1 of 798 requests) in a rehearsal on staging before the pipeline was first

pushed. After the fix in build #4, the following releases served **23,472 requests without a single failure**.

Table 8.4 – Requests during production releases

Build	Release	Requests	Failed	Switch
#3	0b7bd0aa (blue → green)	4,616	1 (HTTP 404)	0.5 s
#4	50f6f382 (green → blue)	4,522	0	0.6 s
#5	b9b6e55c (blue → green)	4,764	0	0.4 s
#6	ef8e78d1 (green → blue)	4,756	0	0.4 s
#7	d062da36 (rolled back)	4,732	0	0.4 s + 0.4 s back
#8	a24d61de (blue → green)	4,698	0	0.4 s



Figure 8.5 – Build #4: blue-green release to production and the smoke test

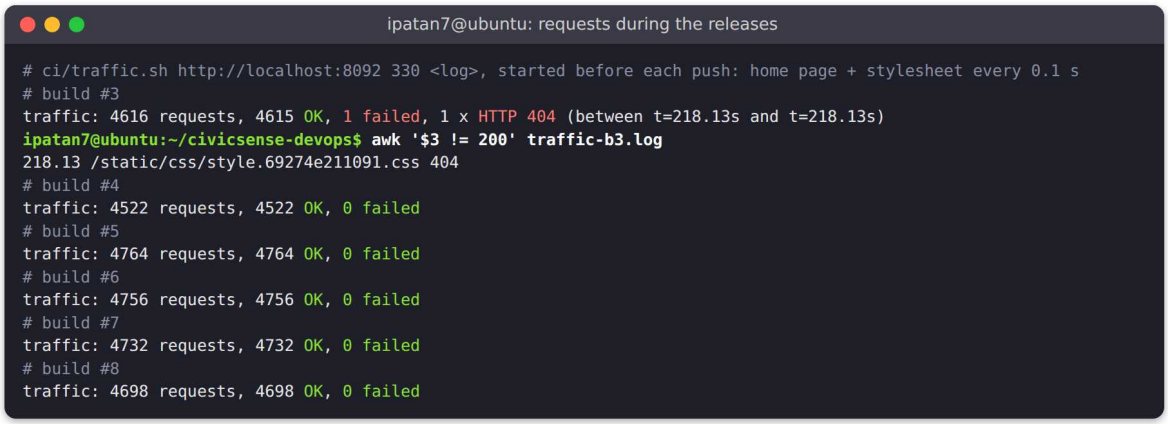


Figure 8.6 – Requests counted during each production release

8.8 Automatic Rollback

Build #7 tested the rollback with a realistic mistake: the production environment file was changed to trust only `https://` origins, as someone might do while preparing for TLS before

TLS exists. Such a change cannot be seen by unit tests, and staging, with its own correct file, passed. In production the new slot started, passed its health check and its direct checks, and nginx switched to it. The smoke test then failed at the fifth check: the login returned **403** because Django's CSRF origin check rejected the browser's `http://` origin. `ci/deploy.sh` switched back to the previous slot in **0.4 s**, recorded the failed release, marked the slot so that it can never be selected by a manual rollback, and failed the build (Figure 8.7). The switch and the rollback were logged in the same second, so the faulty release was live for less than one second; the 4,732 page and stylesheet requests during this build all succeeded, and production went on serving the previous release (`ef8e78d1`) until the revert was released as build #8.

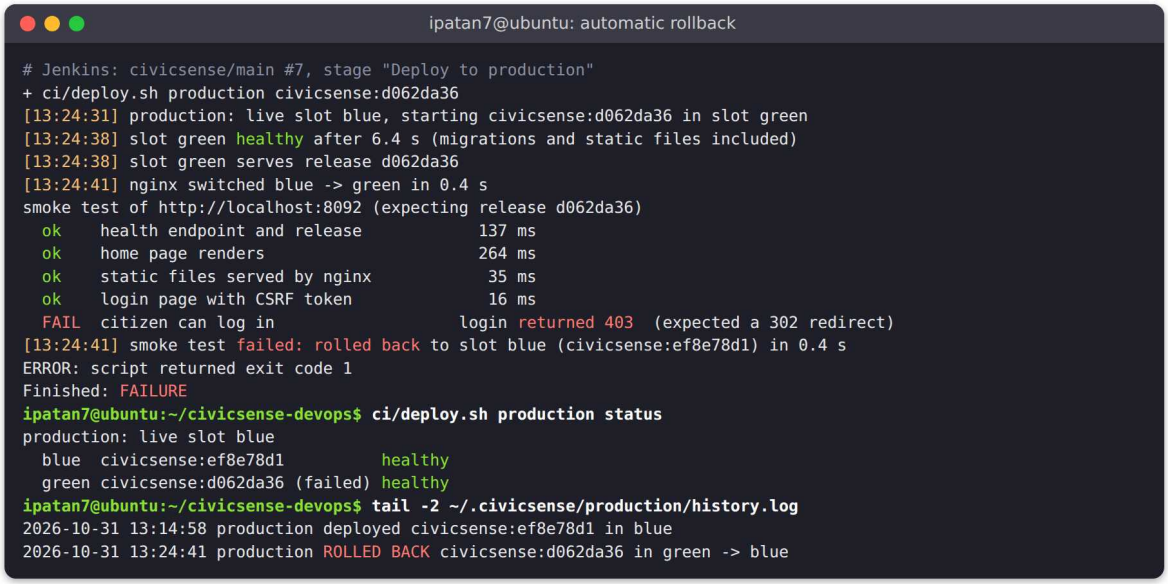


Figure 8.7 – Build #7: the login check fails after the switch and production rolls back by itself

8.9 Manual Rollback

A rollback can also be started by hand, for example when a problem is noticed hours after a release. With traffic running, `ci/deploy.sh production rollback` switched production from the green slot back to the blue one in **0.4 s**, and a second call switched it forward again in another 0.4 s. Each 30-second measurement window carried 438 requests, and none failed (Figure 8.8).

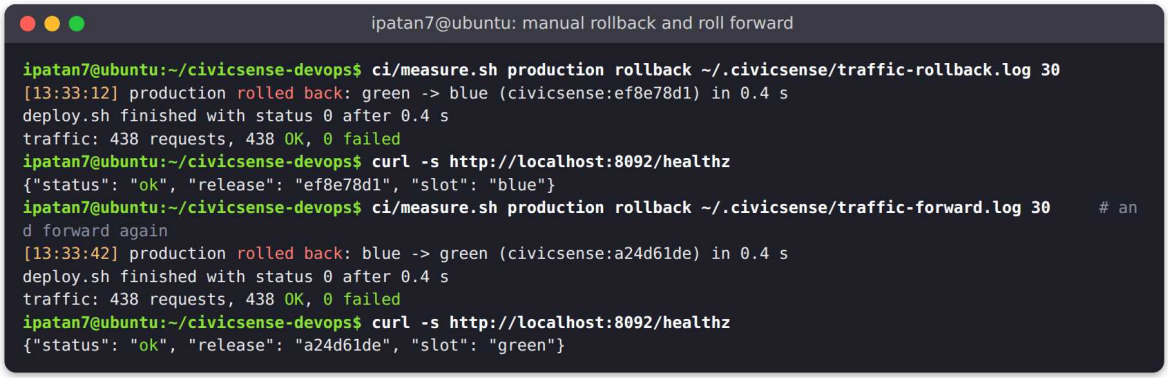


Figure 8.8 – Manual rollback and roll forward under traffic

8.10 Lead Time and Feedback

Without a webhook, Jenkins found each new commit on its one-minute scan **17 to 69 s** after it reached GitHub (mean 47 s). From the moment a commit reached GitHub to the moment production served it took **209 to 260 s** for the seven successful releases (mean 229 s), all of it without a manual step. A failing test was reported on the feature branch 152 s after its build started, and the faulty production configuration was rolled back 269 s after it was pushed (13:20:12 to 13:24:41).

Table 8.5 – Delivery measurements in the style of the DORA metrics

Measure	Result on 31 October 2026
Deployments to production	8 attempted (#2–#9), 7 successful, 1 rolled back automatically
Lead time for changes	209–260 s from push to production (mean 229 s)
Change failure rate	1 of 8 production deployments (the deliberate test in #7)
Time to restore	0.4 s automatic rollback; 0.4 s manual rollback
Failed user requests during releases	1 of 4,616 before the static-files fix, 0 of 23,472 after it

8.11 Summary of Results

Table 8.6 – Summary of measured results

Test	Result
Jenkins controller from files	77 plugins installed in 21 s; up in 9 s with the job created
Pipeline duration (normal release)	167–223 s, mean 182 s
Tests in the image on MySQL 8.4	41 / 41 passed; 82 % statement coverage
Static analysis	0 flake8 findings; bandit 25 → 24 low-severity findings
Dependency audit	7 packages, no known vulnerabilities
Release of one environment	about 12 s; nginx switch 0.4–0.6 s

Test	Result
Requests during releases after the fix	23,472 sent, 0 failed
Faulty configuration in production	Detected by the smoke test, rolled back in 0.4 s
Push to production	209–260 s, no manual step

8.12 Defects Found and Fixed

Building and running the pipeline exposed defects in the pipeline itself, in the deployment and in the application. Table 8.7 lists them; each was fixed in its own commit.

Table 8.7 – Defects found and fixed

Defect	Symptom and cause	Fix
Jenkins refused its configuration	*Failed ConfigurationAsCode.init*: `excludeClientIPFromCrumb` no longer exists in Jenkins 2.580	Setting removed; the default crumb issuer is used
Scripts not executable	Build #1: *ci/lint.sh: Permission denied*. On the NTFS drive Git had `core.fileMode=false` and stored the scripts as 100644	`git update-index --chmod=+x`; all 10 scripts now 100755
Smoke test saw the old release	Rehearsal: the check right after `nginx -s reload` was answered by an old worker, and the release was rolled back although it was good	The switch waits until the public address answers from the new slot
Static file 404 during a release	Build #3: 1 of 4,616 requests failed; `collectstatic --clear` emptied the shared volume	`--clear` removed; hashed names let versions coexist
Demo passwords in production	Found by a security review: the first deployment loaded the demo accounts everywhere, with `--reset`	Demo data only where LOAD_DEMO_DATA=1, never reset; generated smoke-test account
Password in process list	The smoke-test password was a command-line argument of `docker compose exec` and curl	Passed by name from the environment and on stdin
Predictable tracking ids	bandit B311: `random.choices` for an id that opens a complaint	`secrets.choice` (build #6)
Lint never failed in GitHub Actions	The step ended in `|| true`	Same rule as Jenkins: E9/F63/F7/F82 fail
Unreadable release in the footer	Seen in the production screenshot: `text-muted` gave dark grey on the dark-blue footer; the test passed because the text was in the page	`opacity-75` (build #9)

CHAPTER 9: SCREENSHOTS

The screenshots below were taken on 31 October 2026 from the running Jenkins controller, from GitHub and from the production environment.

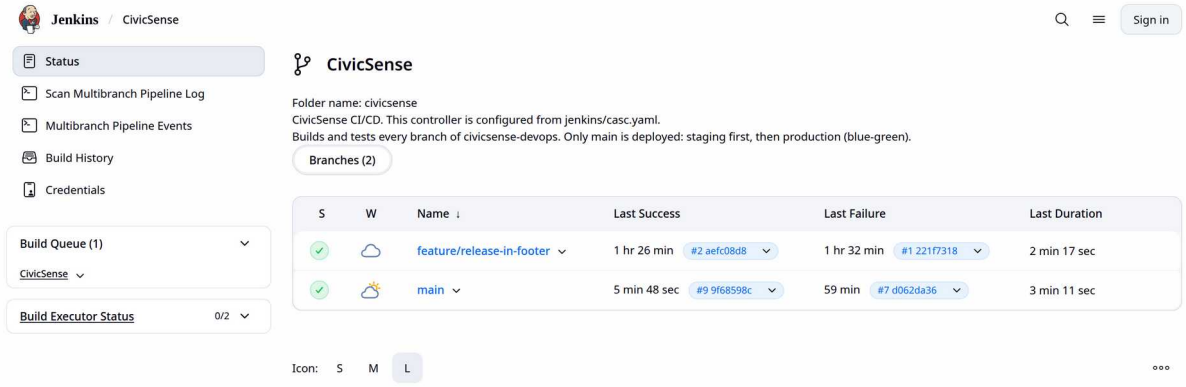


Figure 9.1 – Jenkins dashboard: the multibranch job created from casc.yaml, with its two branches

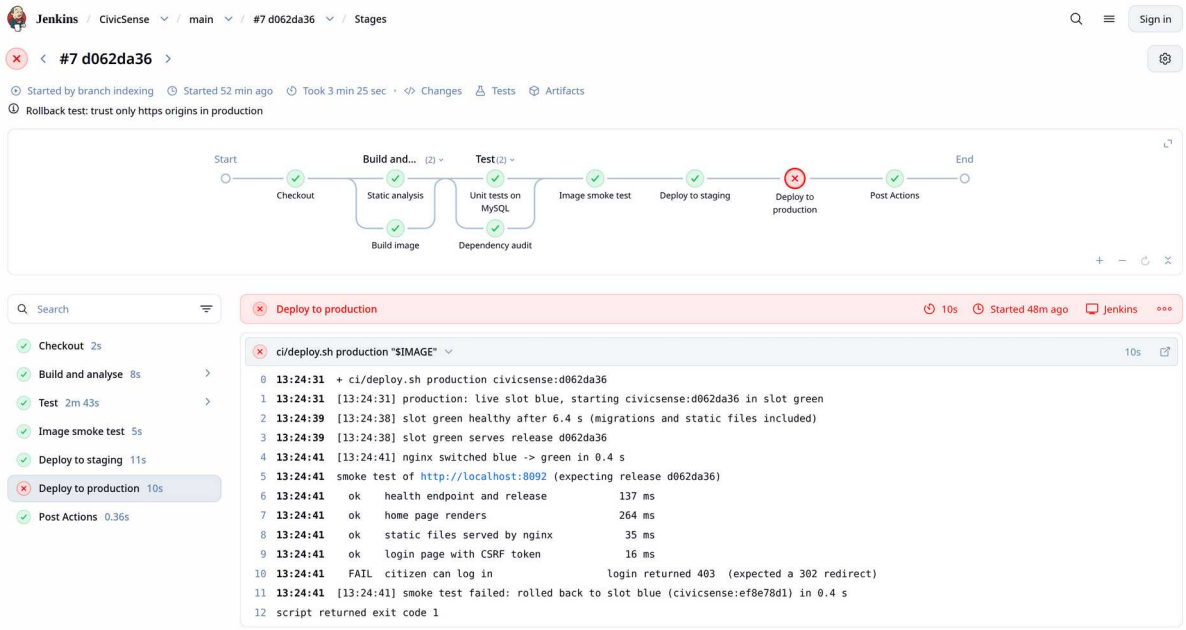


Figure 9.2 – Pipeline graph of build #7: parallel stages, and the failed production deployment

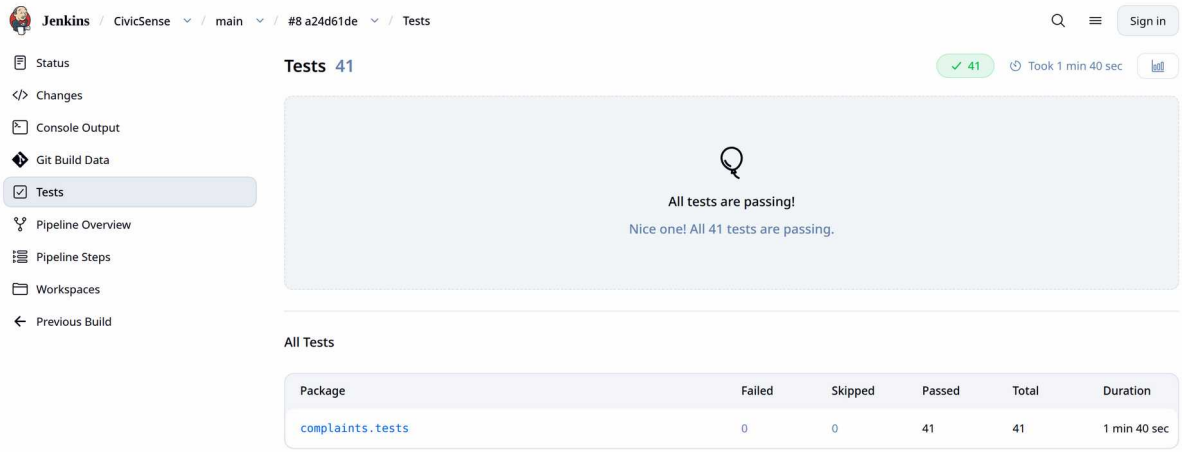


Figure 9.3 – JUnit test results of a main build: 41 tests



Figure 9.4 – The feature branch job: the failing test-first build and the passing build

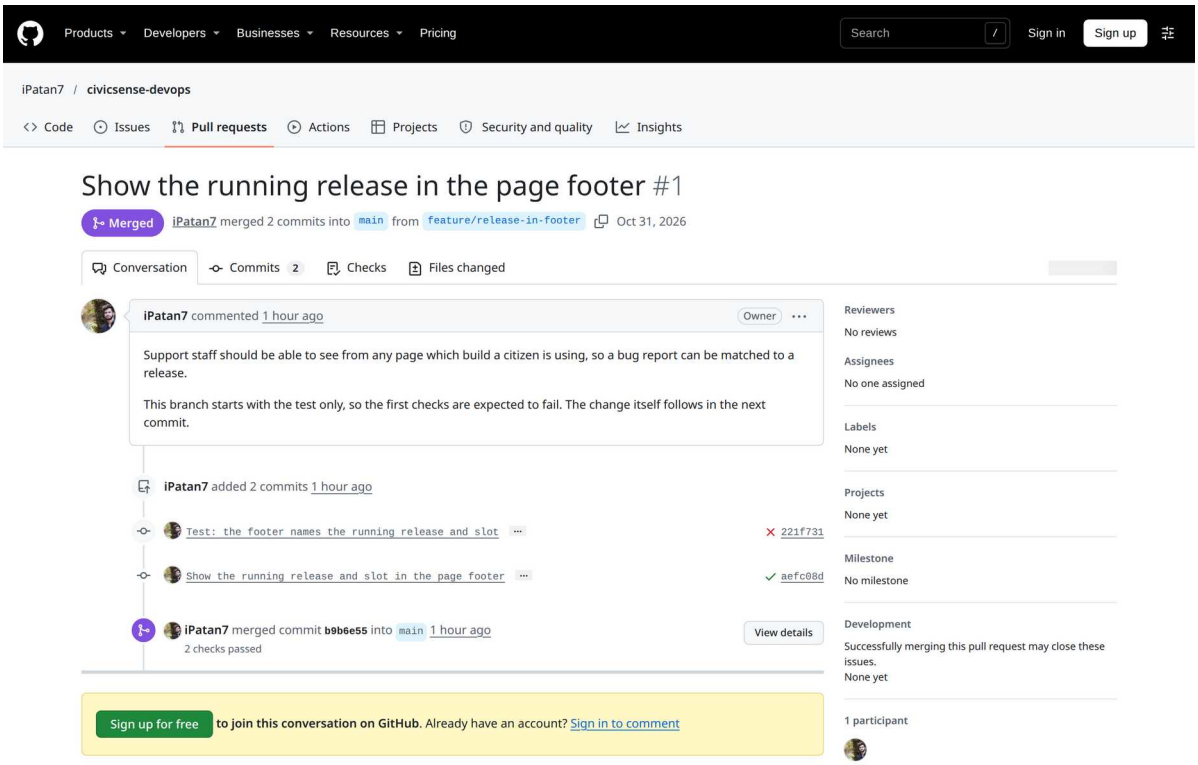


Figure 9.5 – Pull request #1 on GitHub with its checks, merged into main

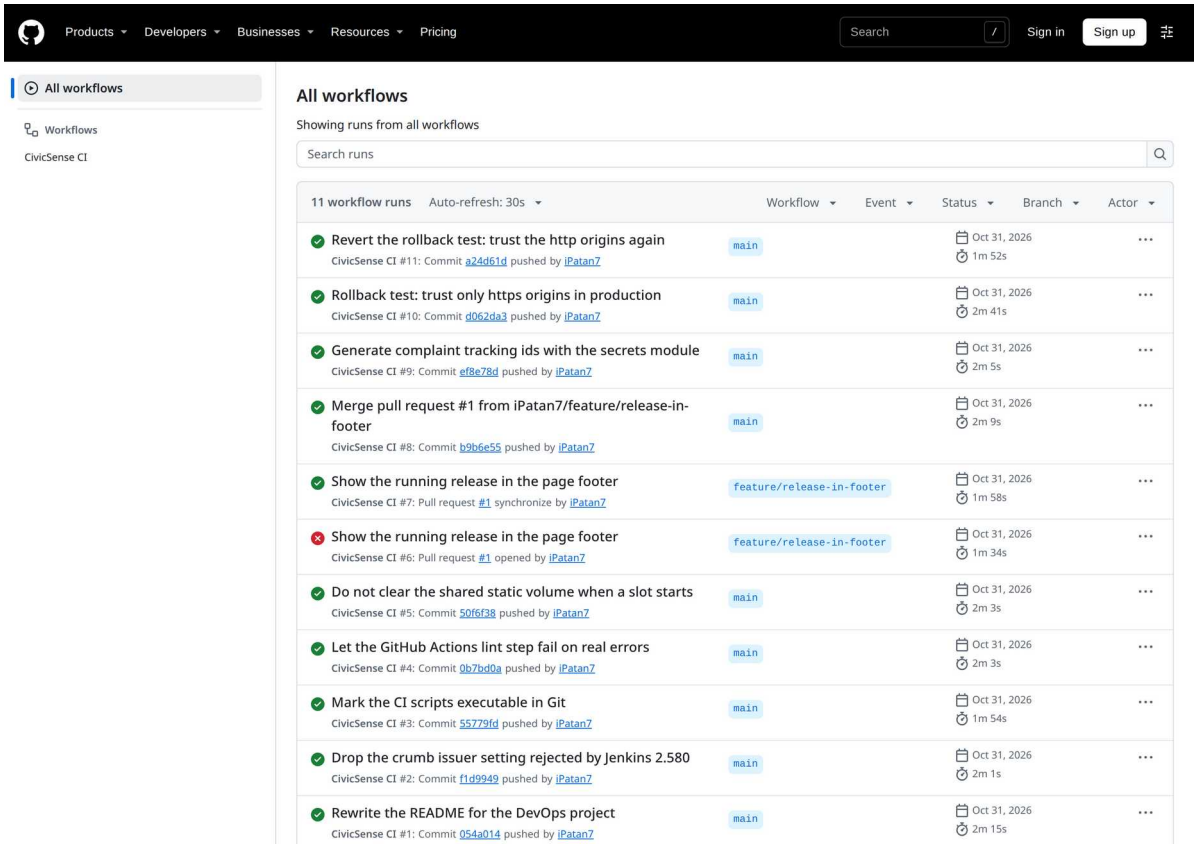


Figure 9.6 – GitHub Actions runs of the repository

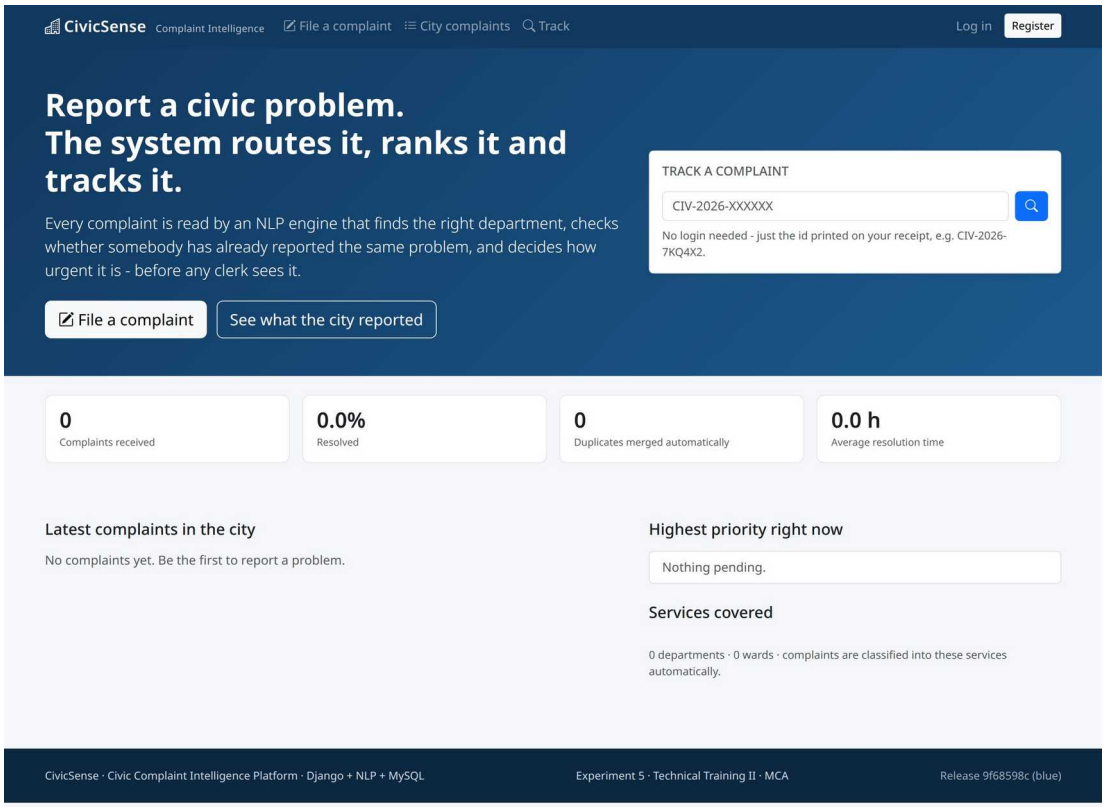


Figure 9.7 – Production portal after build #9: no demonstration data, and the footer names the release and slot that served the page

CHAPTER 10: CONCLUSION

This project turned the release of CivicSense from a manual procedure into an automated pipeline. A declarative Jenkinsfile builds and tests every branch: static analysis beside the image build, 41 tests inside the production image against MySQL 8.4 beside a dependency audit, and a smoke test of the image. Changes on `main` go on to staging and production through a blue-green switch, and a release that fails its smoke test after the switch is undone automatically. The Jenkins controller itself, with its plugins, its security settings and its job, is rebuilt from three files in the repository.

The measurements show that the pipeline does what it promises. A normal release reached production about four minutes after the push, without a manual step. After one defect was found and fixed, releases served 23,472 requests during their switches without a single failure. A failing test on a feature branch stopped that branch in Jenkins and on the pull request, and nothing reached any environment. A faulty production configuration, invisible to unit tests and to staging, was caught by a real login in the smoke test and rolled back in 0.4 s, before any user could notice.

The most instructive results were the defects. A Jenkins setting that no longer existed, scripts that lost their executable bit on an NTFS drive, an nginx reload that returns before it takes effect, and a static-files command that briefly broke the live site during a release – none of these could be seen on the developer's machine, and each was exposed by running the delivery process for real. That is the practical argument for DevOps automation: the release procedure is tested every time it runs, instead of being trusted.

The project met its objectives: continuous integration with parallel stages and quality gates, tests on the real database inside the real image, a feature-branch workflow checked by two independent systems, continuous deployment to two environments without downtime, automatic and manual rollback, configuration management with secrets outside Git, and a CI server defined entirely as code.

CHAPTER 11: FUTURE SCOPE

- **Webhooks instead of scanning** – with a public address (or a relay such as smee.io), a GitHub webhook would start each build within seconds instead of up to a minute.
- **Agents** – run builds on separate Jenkins agents, for example Docker-based agents created per build, so the controller only schedules.
- **Faster tests** – bake coverage and the XML test runner into a test image instead of installing them in every run (15–75 s), and split the 41 tests over parallel workers; the test stage is three quarters of every run.
- **Image registry and signing** – push every tagged image to a registry, scan it with Trivy, sign it, and deploy by digest.
- **Canary releases and metrics** – send a small share of traffic to the new slot first and decide automatically from error rates collected by Prometheus.
- **Database migrations** – an expand-and-contract policy, checked by the pipeline, so that both slots always work with the current schema.
- **Approval gate** – an `input` step before production for continuous delivery instead of continuous deployment, where a business decision is needed.
- **Notifications** – report failed builds and rollbacks to the team through e-mail or a chat channel.

REFERENCES

[1] Jenkins project, "Pipeline" and "Pipeline Syntax", https://www.jenkins.io/doc/book/pipeline/ (accessed October 2026).

[2] Jenkins project, "Branches and Pull Requests" (multibranch pipelines), https://www.jenkins.io/doc/book/pipeline/multibranch/.

[3] Jenkins project, "Configuration as Code" plugin, https://plugins.jenkins.io/configuration-as-code/.

[4] Jenkins project, "Job DSL" plugin and API viewer, https://plugins.jenkins.io/job-dsl/.

[5] Jenkins project, "Plugin Installation Manager Tool", https://github.com/jenkinsci/plugin-installation-manager-tool.

[6] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.

[7] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook, IT Revolution Press, 2016.

[8] N. Forsgren, J. Humble and G. Kim, Accelerate: The Science of Lean Software and DevOps, IT Revolution Press, 2018.

[9] M. Fowler, "BlueGreenDeployment", https://martinfowler.com/bliki/BlueGreenDeployment.html.

[10] S. Chacon and B. Straub, Pro Git, 2nd edition, Apress, https://git-scm.com/book.

[11] GitHub, "About pull requests", "Workflow syntax for GitHub Actions" and "About service containers", https://docs.github.com/.

[12] Docker Inc., "Docker Compose: use --wait" and "Compose file reference", https://docs.docker.com/compose/.

[13] NGINX, "Controlling nginx" (graceful reload) and "ngx_http_upstream_module", https://nginx.org/en/docs/.

[14] Django Software Foundation, "Cross Site Request Forgery protection" and "Testing in Django", https://docs.djangoproject.com/en/6.1/.

[15] PyCQA, "flake8" and "bandit" documentation; PyPA, "pip-audit", https://github.com/pypa/pip-audit.

[16] A. Wiggins, "The Twelve-Factor App", https://12factor.net/.

[17] M. I. Patan, "civicsense-docker: CivicSense containerised with Docker", GitHub, 2026, https://github.com/iPatan7/civicsense-docker.

[18] M. I. Patan, "civicsense-devops: CivicSense delivered automatically – pipeline, deployment scripts and project report", GitHub, 2026, https://github.com/iPatan7/civicsense-devops.

APPENDIX A: RUNNING AND DEMONSTRATING THE PROJECT

The steps below rebuild the whole setup on a Linux machine with Docker, Git and Java 21 or newer, and show every result described in this report.

A.1 Start Jenkins from the repository

```
git clone https://github.com/iPatan7/civicsense-devops.git
cd civicsense-devops
export JENKINS_ADMIN_PASSWORD='choose-a-password'
jenkins/run-jenkins.sh         # Jenkins with its job, on http://localhost:8090
```

Within a minute the `CivicSense` job scans the repository and builds `main`: the first build deploys staging (http://localhost:8091, with demo data) and production (http://localhost:8092, without).

Table A.1 – Accounts

Where	User	Password
Jenkins	admin	the value of JENKINS_ADMIN_PASSWORD
Staging portal	citizen1, officer, admin	civic123, civic123, admin123 (demo data)
Smoke-test account	smoke-test	generated, in ~/.civicsense/<env>/secrets.env

A.2 Run any stage by hand

```
docker build -t civicsense:try .
ci/lint.sh                             # flake8 and bandit, reports in reports/
ci/test.sh civicsense:try              # 41 tests on a throw-away MySQL 8.4
ci/audit.sh civicsense:try             # pip-audit of the image's packages
ci/smoke-image.sh civicsense:try       # the image boots and is not root
```

A.3 Release, roll back and measure

```
ci/deploy.sh staging civicsense:try                # blue-green release
ci/deploy.sh staging status                        # which slot is live
ci/deploy.sh staging rollback                      # back to the other slot
ci/smoke.sh http://localhost:8091                  # the 7 checks, demo account
ci/measure.sh staging rollback /tmp/traffic.log 30  # rollback under traffic
```

A.4 Show the branching workflow

```
git switch -c feature/my-change
# ... commit a test and the change ...
git push -u origin feature/my-change      # Jenkins builds the branch within a minute
gh pr create --fill                        # GitHub Actions checks the pull request
gh pr checks                               # both must be green
gh pr merge --merge                        # main is built and released
```

A.5 Stop everything

```
for env in staging production; do
```

```
  docker compose -p civic-$env -f deploy/compose.yml --env-file deploy/env/$env.env \
    --env-file ~/.civicsense/$env/secrets.env down        # add -v to delete the data
done
# Jenkins: stop the run-jenkins.sh process (Ctrl+C)
```